Securing the Vibe Coding Era

Local-First Security Gates for AI-Generated Code

Mladen Maric

SREday Amsterdam · 2 November 2026



It looked like a small change

Illustrative operations-portal endpoint

```
task = request.args["task"]
subprocess.run(task, shell=True)
```

Whose input? Whose authority?

Application security through the lifecycle

Design

Assets
Trust boundaries

Build + test

Code + dependencies
Permissions + behavior

Operate

Patch
Observe + respond

Allowed in staging does not mean allowed in production.

Source and running behavior

SAST

Inspect program material

DAST

Exercise a running service

Different inputs, different evidence. ZAP is one project to explore for DAST.

Inside a running application

IAST

Observe exercised paths

RASP

Detect and attempt to block

Instrumentation adds runtime context. Coverage and operational cost still matter.

Mobile testing and fuzzing

MAST

A domain: mobile apps
Static + dynamic + manual

Fuzzing

A technique: many inputs
Look for failing behavior

A target domain and a testing method can be used together.

There is more than application code

dependencies	Known vulnerable versions
credentials	Secret-looking material
deployment files	Risky configuration

SBOM = recognized component inventory

0

findings

One required scanner did not finish.

Would you allow this push?

What stays under your control?

Source you can inspect

Execution you can control

Maintenance you still own

The projects behind the checks

OpenGrep Bearer

Gitleaks TruffleHog

Syft Grype Trivy Checkov

Open-source and source-available projects. Each retains its own license.

Code and credential tools

Tool	Project capabilities	Lomo integration	Design choice
OpenGrep	SAST: rules + flows	Reviewed rules → SARIF	Inspectable checks
Bearer	SAST + sensitive data	Security report → findings	Actionable code risks
Gitleaks	Secrets: Git, files, stdin	Captured trees → SARIF	One source identity
TruffleHog	Secrets + verification	Files → redacted findings	Local source analysis

Selected modes: identified inputs, prepared data and explainable results.

OpenGrep + Bearer

A risky operation

```
subprocess.run(task, shell=True)
```

A sink match still needs input and execution context.

Gitleaks + TruffleHog

```
CI_SERVICE_TOKEN = "[REDACTED]"
```

Potential secret ≠ verified credential

Lomo disables remote credential verification.

Components and configuration tools

Tool	Project capabilities	Lomo integration	Design choice
Syft	SBOM + metadata	Directory → CycloneDX	Inventory once, reuse
Grype	SCA: known CVEs	Same-scan SBOM → matches	Traceable input
Trivy	SCA, secrets, IaC, licenses	Vuln + secret + config	Broad source checks
Checkov	IaC policy; SCA modes	K8s / Terraform / Dockerfile	Deployment intent

Selected modes: identified inputs, prepared data and explainable results.

Inventory before advisory matching



Trivy provides overlapping dependency checks

Package presence does not establish exploitability.

Checkov + Trivy

```
securityContext:
  privileged: true
```

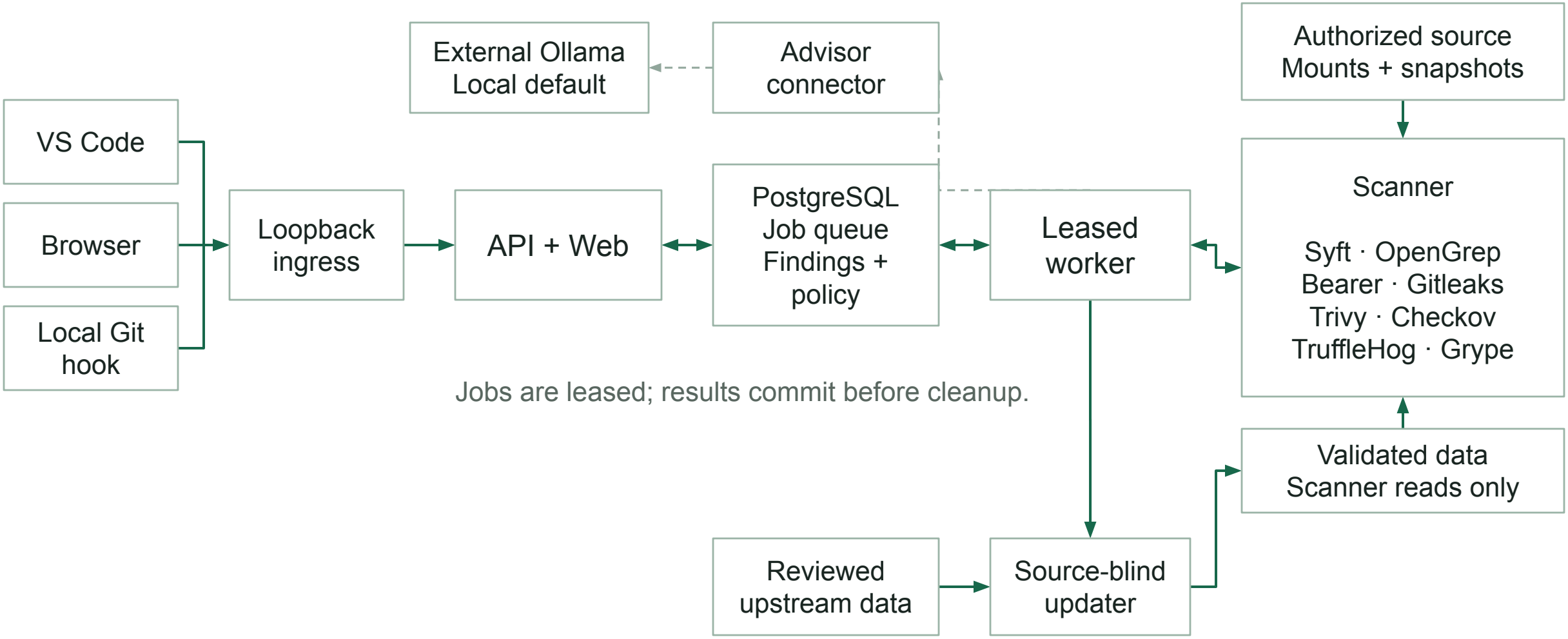
Declared permissions deserve review

A diagnostic DaemonSet copied into production deserves another review.

A toolbox I would have liked to have

A starting point for the next internal project

Inside Lomo CodeSecurity



Explicit source access. Durable results. Advice never decides the gate.

Findings

Control plane healthy

source
Entire registered codebase · Scan #7 · full
▸ Source root

Status **completed**
Gate **fail**
Active Findings **4**

Push protection **No matching protection record**
Manage push protection
A passing scan gate does not verify installed push protection.

#7 · completed
Search findings

▸ Filters and scanner counts
Clear filters | Scan Advisor | ▸ Why?

critical 2 | high 2 | medium 0 | low 0 | info 0

4 findings
4 active · 0 non-blocking · 0 ignored
Select visible

▸ No findings selected · Review actions

S7-F001 critical open — trivy
AWS Access Key ID
app.py:9:1
Secret exposure · aws-access-key-id

S7-F002 critical open — trivy
GitHub Personal Access Token
app.py:10:1
Secret exposure · github-pat

S7-F003 high open — opengrep
A Python subprocess with shell=True invokes a command...
app.py:25:14-25:58
Code issue · kodesec-python-subprocess-shell

S7-F001 critical

AWS Access Key ID

Secret exposure · Detected by trivy · Rule aws-access-key-id

Open source in VS Code | IDE | Run Advisor | Lifecycle

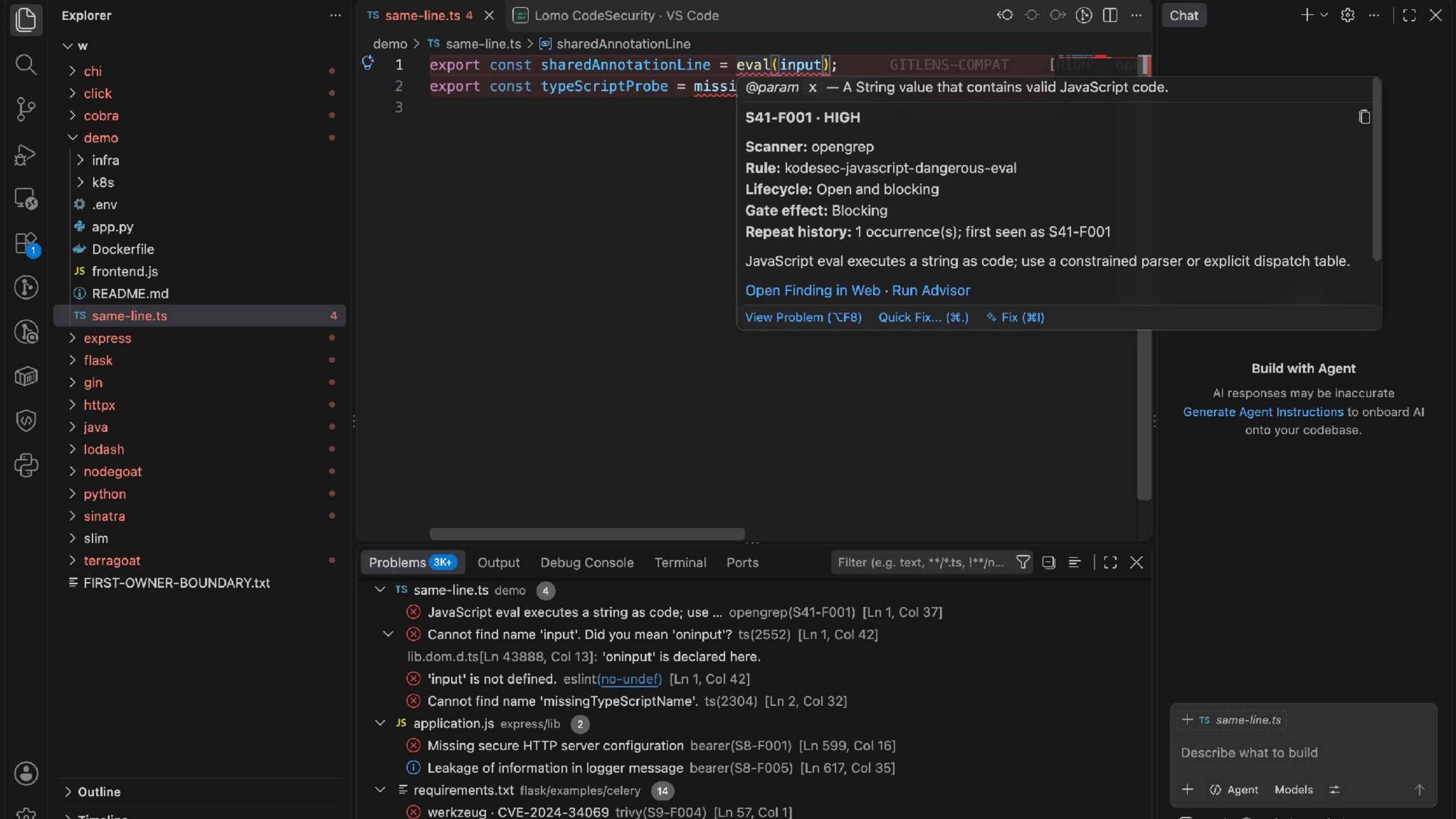
This finding belongs to an immutable Git snapshot, not the current checkout. Opening historical snapshot source in VS Code is not supported yet; a matching filename could contain different code. Review the retained finding details and snippet below.
Lifecycle **open** Gate effect **Blocking**

Evidence app.py:9:1
Copy evidence

```
  7  app = Flask(__name__)
  8
> 9  AWS_ACCESS_KEY_ID = "[REDACTED]"
 10  GITHUB_TOKEN = "[REDACTED]"
 11  PAYMENT_API_KEY = "[REDACTED]"
 12
 13
 14  @app.get("/lookup")
 15  def lookup_user():
 16      user_id = request.args.get("id", "1")
```

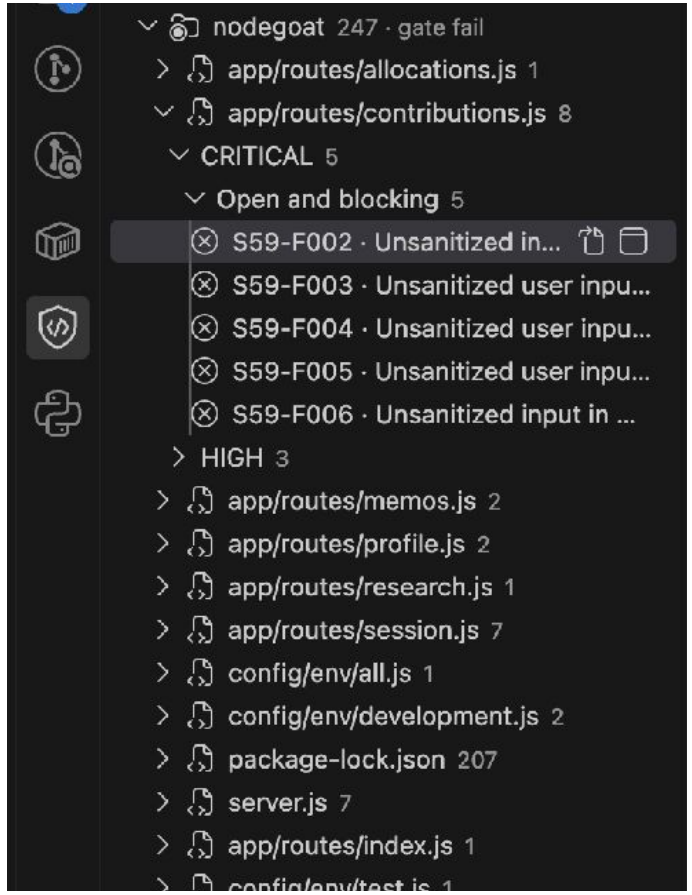
More code
Max height

▸ Scanner description

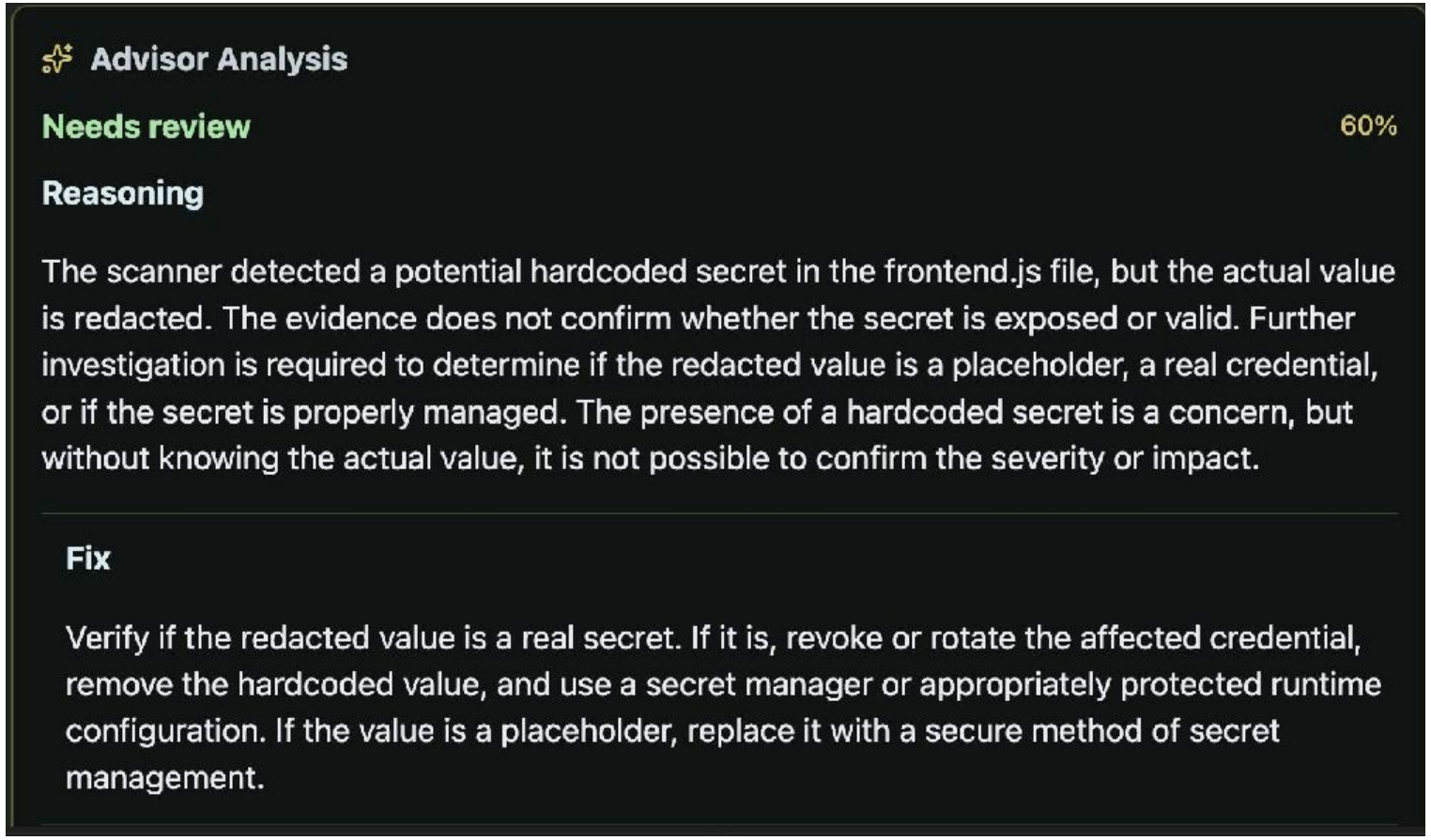

Explorer
same-line.ts 4
Lomo CodeSecurity · VS Code
Chat
demo > same-line.ts > sharedAnnotationLine
1 export const sharedAnnotationLine = eval(input);
2 export const typeScriptProbe = missi
3
@param x — A String value that contains valid JavaScript code.
S41-F001 · HIGH
Scanner: opengrep
Rule: kodesec-javascript-dangerous-eval
Lifecycle: Open and blocking
Gate effect: Blocking
Repeat history: 1 occurrence(s); first seen as S41-F001
JavaScript eval executes a string as code; use a constrained parser or explicit dispatch table.
Open Finding in Web · Run Advisor
View Problem (⌥F8) Quick Fix... (⌘.) Fix (⌘I)
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
Problems 3K+
Output
Debug Console
Terminal
Ports
same-line.ts demo 4
JavaScript eval executes a string as code; use ... opengrep(S41-F001) [Ln 1, Col 37]
Cannot find name 'input'. Did you mean 'oninput'? ts(2552) [Ln 1, Col 42]
lib.dom.d.ts[Ln 43888, Col 13]: 'oninput' is declared here.
'input' is not defined. eslint(no-undef) [Ln 1, Col 42]
Cannot find name 'missingTypeScriptName'. ts(2304) [Ln 2, Col 32]
application.js express/lib 2
Missing secure HTTP server configuration bearer(S8-F001) [Ln 599, Col 16]
Leakage of information in logger message bearer(S8-F005) [Ln 617, Col 35]
requirements.txt flask/examples/celery 14
werkzeug · CVE-2024-34069 trivy(S9-F004) [Ln 57, Col 1]
same-line.ts
Describe what to build
Agent
Models
Outline
Timeline

Find the evidence. Ask for context.

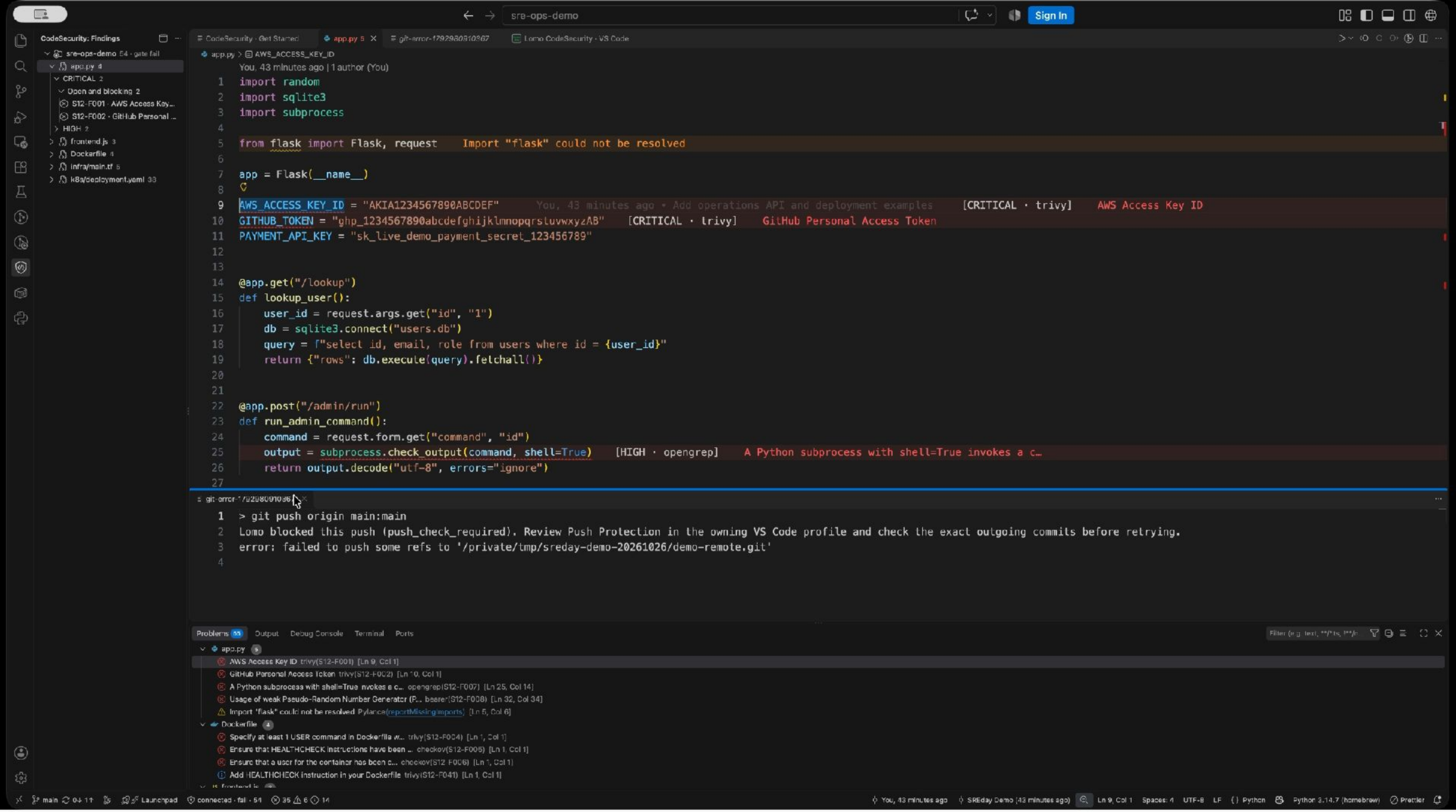
Findings in VS Code



Advisor review



Optional advice. Scanner evidence still decides the gate.



The push is blocked, and the remote still points to the same commit.

The critical findings remain visible.

A small toolbox for your own projects

Run the scanners on a repository you maintain.

Review the findings before you enable blocking.

Fork the code and adapt it to your team.

Keep CI and server-side checks for team-wide enforcement.

Evidence before push

- https://cs.lomo.dev
- touch@lomo.dev
- lomolabs/lomo-codesecurity
- in/mladenmaric
- @lomolabs

